

Relatório do Trabalho Prático 2

Programação Paralela em GPU

Fernanda das Neves Merqueades Santos e Kauê Peixoto Fernandes

25 de maio de 2026

Resumo

Este trabalho apresenta uma implementação paralela do método iterativo de Jacobi utilizando CUDA para execução em GPU. O objetivo foi comparar o desempenho da versão em GPU com implementações sequencial e paralela utilizando OpenMP na resolução de sistemas lineares. A abordagem em CUDA explorou o paralelismo massivo da GPU por meio da execução de kernels responsáveis pelo cálculo das iterações e da norma do erro. Os resultados obtidos mostraram ganhos significativos de desempenho, alcançando speedup superior a 70 vezes em relação à versão sequencial. Também foi analisado o comportamento de convergência do método, observando-se a redução progressiva da norma do erro ao longo das iterações.

1 Introdução

Os sistemas lineares são amplamente utilizados em diversas áreas da computação moderna, como processamento de imagens, visão computacional e inteligência artificial, seja de forma direta na resolução de problemas numéricos, seja como base para modelos matemáticos empregados em algoritmos. Com o crescimento do volume de dados e da complexidade desses problemas, tornou-se inviável resolver esses sistemas de forma puramente sequencial.

Assim, o uso de múltiplos processadores, GPUs e infraestruturas distribuídas tem se tornado essencial para alcançar desempenho adequado, convertendo o paralelismo em uma estratégia fundamental para melhor aproveitamento dos recursos computacionais disponíveis como apontam [Grama et al., 2003, Wilkinson and Allen, 2005].

Dentre os métodos utilizados para resolver sistemas lineares, destacam-se os métodos iterativos, como o método de Jacobi descrito em [Burden and Faires, 2011]. Esse método trabalha com a geração de aproximações sucessivas para a solução do sistema, utilizando os valores da iteração anterior até que o erro entre duas iterações consecutivas seja suficientemente pequeno. Uma das principais características do método é que cada equação do sistema pode ser calculada independentemente das demais dentro de uma mesma iteração, sendo especialmente adequado para problemas de grande porte, nos quais a obtenção da solução exata pode ser computacionalmente custosa.

A utilização de GPUs permite executar milhares de threads simultaneamente, possibilitando ganhos de desempenho superiores aos obtidos em arquiteturas multicore tradicionais, especialmente em problemas numéricos com alto grau de paralelismo.

No Trabalho Prático 1 foi desenvolvida uma versão paralela do método utilizando OpenMP, explorando paralelismo em arquiteturas de memória compartilhada por meio do uso de múltiplas threads na CPU. Já no Trabalho Prático 2, a proposta consiste em adaptar o algoritmo para execução em GPU utilizando CUDA, explorando paralelismo massivo nessas arquiteturas.

2 Ambiente Experimental

Os experimentos foram realizados em um sistema com as seguintes configurações:

- **Processador:** AMD Ryzen 5 5500
- **Número de núcleos e threads:** 6 núcleos e 12 threads
- **Memória RAM:** 16 GB DDR4 2666 MHz CL20
- **GPU:** NVIDIA RTX 3060 com 12 GB GDDR6
- **Sistema Operacional:** Windows 11 versão 25H2
- **Compilador CUDA (NVCC):** CUDA compilation tools, release 13.2, V13.2.78
- **Compilador C/C++:** MSVC versão 19.50.35730 para x64
- **API utilizada:** CUDA

3 Implementação

O trabalho foi desenvolvido em linguagem C++, utilizando CUDA para a execução paralela na GPU. O código foi dividido em múltiplos arquivos com o objetivo de melhorar a organização e facilitar a manutenção.

O arquivo `main.cpp` é responsável pela criação do sistema linear, execução das versões sequencial e paralela do método de Jacobi, medição dos tempos de execução e cálculo do speedup entre CPU e GPU.

A implementação sequencial do método foi separada no arquivo `sequencial.cpp`, contendo a função responsável pelas iterações do método de Jacobi utilizando apenas a CPU.

As funções auxiliares foram organizadas em `utils.cpp`. Essa função inclui a geração da matriz diagonal dominante utilizada nos testes, o cálculo da diferença entre os resultados obtidos na CPU e na GPU e a conversão da matriz bidimensional para um vetor unidimensional, pois a GPU trabalha de forma mais eficiente com vetores lineares na memória global.

A implementação paralela utilizando CUDA foi organizada em kernels executados na GPU. O kernel principal realiza o cálculo das novas aproximações do método de Jacobi, atribuindo uma thread para cada equação do sistema linear. Além disso, foi

implementado um segundo kernel responsável pelo cálculo do erro entre iterações consecutivas, utilizado para verificar a convergência do método iterativo.

O `kernel_jacobi` é responsável pelo cálculo paralelo das equações do sistema linear. Cada thread da GPU calcula uma posição do vetor solução x , permitindo que várias equações sejam processadas simultaneamente.

O índice global de cada thread é calculado utilizando os identificadores de bloco e thread fornecidos pelo CUDA:

$$i = blockIdx.x \times blockDim.x + threadIdx.x$$

Além da memória global, foi utilizada memória compartilhada no kernel de cálculo do erro. Essa memória permite comunicação mais rápida entre threads pertencentes ao mesmo bloco, reduzindo acessos à memória global durante a operação de redução paralela.

Inicialmente, os dados do sistema linear são gerados na CPU. Em seguida, a matriz e os vetores são transferidos para a memória da GPU utilizando a função `cudaMemcpy`.

Ao final do processamento, o vetor solução final é transferido novamente para a CPU, permitindo comparar os resultados obtidos pela implementação sequencial e pela implementação paralela em GPU.

3.1 Decisões de Projeto

Nos experimentos realizados foram utilizados diferentes tamanhos de bloco em cada execução, variando manualmente entre 64 e 1024 threads por bloco, com o objetivo de analisar o impacto dessa configuração no desempenho da GPU.

Durante a implementação, foi adotado o valor de 256 threads por bloco, pois esse tamanho representa um valor comumente utilizado em aplicações CUDA, oferecendo um bom equilíbrio entre paralelismo e utilização dos recursos da GPU.

Cada thread foi responsável pelo cálculo de uma equação do sistema linear, permitindo que múltiplas linhas da matriz fossem processadas simultaneamente.

Essa abordagem garante que todas as posições do vetor solução sejam processadas, mesmo quando o tamanho do problema não é múltiplo exato do número de threads por bloco.

A matriz do sistema foi armazenada em um vetor unidimensional na memória global, permitindo acesso direto pelas threads da GPU.

A operação `atomicAdd` foi utilizada para acumular o erro global calculado pelos blocos, evitando condições de corrida durante a atualização da variável compartilhada entre as threads.

Na versão sequencial e paralela, foi utilizada a forma normalizada da norma do erro, dada por:

$$\frac{\|x^{(k+1)} - x^{(k)}\|}{\|x^{(k+1)}\|} < \epsilon$$

Além do critério baseado no erro, também foi utilizado um limite máximo de 5000 iterações para evitar execuções excessivamente longas em casos de não convergência.

4 Resultados

4.1 Desempenho

Os experimentos foram realizados considerando uma matriz com 2500 elementos.

Versão	Configuração	Tempo (s)	Speedup vs Seq	Speedup vs OpenMP
Sequencial	CPU	1023,85	1,00X	-
OpenMP	12 threads	54,97	19,70X	1,00X
GPU	64 threads/bloco	14,50	70,61X	3,79X
GPU	128 threads/bloco	14,47	70,76X	3,80X
GPU	256 threads/bloco	14,67	69,79X	3,78X
GPU	512 threads/bloco	17,91	57,17X	3,07X
GPU	1024 threads/bloco	37,16	27,55X	1,48X

Table 1: Comparação de desempenho entre implementação sequencial, OpenMP e GPU

Os resultados mostram que a utilização da GPU proporcionou um ganho significativo de desempenho em relação às implementações em CPU. A versão sequencial apresentou o maior tempo de execução, enquanto a implementação utilizando OpenMP reduziu consideravelmente esse tempo através do paralelismo em múltiplas threads da CPU. Entretanto, os melhores resultados foram obtidos com CUDA na GPU, alcançando speedup superior a 70 vezes em relação à versão sequencial.

Também foi observado que as configurações utilizando 64, 128 e 256 threads por bloco apresentaram desempenhos muito próximos. Isso indica que, nessas configurações, a GPU já atingia um nível adequado de ocupação e paralelismo. Por outro lado, configurações com 512 e 1024 threads por bloco apresentaram pior desempenho, devido ao aumento do overhead de sincronização e à menor eficiência na utilização dos recursos internos da GPU.

4.2 Convergência

A Tabela 2 apresenta a evolução da norma do erro ao longo das iterações do método de Jacobi executado na GPU.

Iteração	Norma do Erro
0	1
50	0.8552
100	0.8444
200	0.8230
300	0.8018

Table 2: Convergência do método de Jacobi na GPU

Observa-se que a norma do erro diminui progressivamente ao longo das iterações, indicando que o método de Jacobi está convergindo gradualmente para a solução do

sistema linear. A redução do erro ocorre de forma relativamente lenta, característica comum de métodos iterativos como Jacobi, especialmente em sistemas lineares de grande dimensão.

É importante destacar que a utilização da GPU acelera significativamente o tempo de processamento das iterações, porém não altera o comportamento matemático da convergência do método. Dessa forma, a implementação paralela mantém os mesmos critérios de convergência observados na versão sequencial.

4.3 Discussão

A implementação utilizando CUDA apresentou desempenho significativamente superior às versões sequencial e OpenMP, especialmente para sistemas lineares de maior dimensão. Esse resultado está relacionado à capacidade da GPU de executar milhares de threads simultaneamente. Em comparação ao OpenMP, que explora paralelismo em múltiplas threads da CPU, o CUDA oferece um paralelismo massivo mais adequado a problemas com alta decomposição de dados. Entretanto, o ganho de desempenho não ocorre de forma linear, pois ainda existem custos associados à transferência de dados entre CPU e GPU e à sincronização necessária entre as iterações do método.

A configuração dos blocos de threads também influenciou o desempenho da aplicação. Tamanhos intermediários de bloco tendem a apresentar melhor aproveitamento da GPU, enquanto valores muito baixos subutilizam os recursos disponíveis e valores muito altos podem gerar competição por recursos internos da GPU, prejudicando o desempenho.

Outro ponto importante é que o método de Jacobi exige sincronização global entre iterações, o que implica na necessidade de reinicialização de dados e comunicação frequente entre CPU e GPU. Além disso, acessos recorrentes à memória global durante o cálculo das atualizações atuam como gargalo de desempenho.

Embora a GPU ofereça ganhos expressivos de paralelismo, o desempenho final do algoritmo é limitado principalmente pela latência de comunicação, pelo padrão de acesso à memória e pelo número de iterações necessárias para convergência.

Durante o desenvolvimento da implementação em CUDA, algumas dificuldades foram encontradas, principalmente na tentativa de adaptação do código desenvolvido com OpenMP para o modelo de execução da GPU. Também houve problemas no cálculo da norma do erro e no gerenciamento de memória da GPU, especialmente relacionados à liberação correta da memória alocada. Além disso, o aumento da complexidade do código exigiu uma melhor organização da implementação, levando à criação de arquivos auxiliares para facilitar a manutenção e legibilidade do projeto.

5 Conclusão

Neste trabalho foi desenvolvida uma implementação paralela do método de Jacobi utilizando CUDA para execução em GPU. Os resultados obtidos mostraram que a abordagem em GPU apresentou desempenho significativamente superior às versões sequencial e OpenMP, alcançando speedup superior a 70 vezes em relação à execução sequencial.

Os experimentos também mostraram que a configuração do número de threads por bloco influencia diretamente o desempenho da aplicação, sendo observados melhores

resultados para tamanhos intermediários de bloco. Além disso, foi possível verificar que a GPU acelera o processamento das iterações sem alterar o comportamento de convergência do método.

Durante o desenvolvimento, foram encontradas dificuldades relacionadas principalmente à adaptação do código para o modelo de execução em GPU, ao gerenciamento de memória e ao cálculo da norma do erro. Apesar disso, a implementação final apresentou resultados corretos e desempenho satisfatório.

Como melhorias futuras, o projeto pode receber uma reorganização estrutural do código, com melhor separação de responsabilidades, maior modularização e documentação mais detalhada. Também podem ser implementadas seções específicas para entrada de dados, compilação e apresentação dos resultados, além da automatização dos testes utilizando matrizes de diferentes tamanhos e características, como matrizes diagonalmente dominantes e matrizes aleatórias, reduzindo a necessidade de alterações manuais no código durante os experimentos.

References

- [Burden and Faires, 2011] Burden, R. L. and Faires, J. D. (2011). *Numerical Analysis*. Cengage Learning.
- [cppreference,] cppreference. C++ reference.
- [Grama et al., 2003] Grama, A., Gupta, A., Karypis, G., and Kumar, V. (2003). *Introduction to Parallel Computing*. Addison-Wesley, 2 edition.
- [Microsoft,] Microsoft. Visual studio code.
- [MinGW,] MinGW. Minimalist gnu for windows.
- [NVIDIA, 2026] NVIDIA (2026). Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-programming-guide/>. Documentação oficial do CUDA. Acesso em: 25 maio 2026.
- [UNIVESP, 2022] UNIVESP (2022). Computação escalável - frameworks para computação paralela - cuda. <https://www.youtube.com/watch?v=Gm2LlkyEC50>. Videoaula ministrada por Júlio Cezar Estrella. Acesso em: 25 maio 2026.
- [Wilkinson and Allen, 2005] Wilkinson, B. and Allen, M. (2005). *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*. 2 edition.